MINI PROJECT REPORT

TODO LIST APPLICATION USING DEVOPS TOOLS

Academic Year: 2026–2027

Group Members:

1. Pratiksha Jadhav — Student ID: 24107107
2. Bhavesh Bonde — Student ID: 24107046
3. Shravani Karpe — Student ID: 24107047
4. Preshita Patil — Student ID: 24107026

Semester: V

Class / Branch: TE-DS-B

Subject: Agile

Name of Instructor: Prof. Sameer

Date of Performance: 23/10/2026

Date of Submission: 23/10/2026

1. INTRODUCTION

DevOps is a software development approach that combines development and operations practices to improve software delivery, testing, deployment, and maintenance. It encourages automation, collaboration, version control, and continuous monitoring throughout the software development lifecycle.

The Todo List DevOps Mini Project demonstrates a web-based task management application supported by DevOps tools. The project uses Python for backend processing, HTML and CSS for the user interface, Git and GitHub for version control, Docker for containerization, Docker Compose for service configuration, and Prometheus for monitoring configuration. Grafana can be used to visualize monitoring metrics when properly connected to Prometheus.

The project demonstrates how an application can be organized, tested, packaged, and monitored using commonly used software development and operations tools.

2. PROBLEM STATEMENT

Managing daily tasks manually can make it difficult to organize activities, track pending work, and maintain a record of completed tasks. At the same time, developing and running an application in different environments may lead to dependency conflicts and deployment difficulties.

The problem addressed by this project is to develop a Todo List web application and demonstrate the use of DevOps tools for source-code management, automated testing, containerization, service configuration, and monitoring.

3. OBJECTIVES

The main objectives of this project are:

1. To develop a web-based Todo List application.
2. To provide a simple interface for managing daily tasks.
3. To maintain the project source code using Git and GitHub.
4. To manage application dependencies using Python package requirements.
5. To containerize the application using Docker.
6. To configure application services using Docker Compose.
7. To execute automated tests and verify application functionality.
8. To design a Continuous Integration (CI) pipeline for automated testing.
9. To configure Prometheus for monitoring application metrics.
10. To explore Grafana for monitoring visualization.
11. To document the development, testing, and deployment process using screenshots and evidence.

4. APPLICATION REQUIREMENTS AND SYSTEM DESCRIPTION

4.1 Hardware Requirements

- Computer or laptop.
- Minimum 4 GB RAM; 8 GB RAM recommended.
- Sufficient disk space for application files, dependencies, and Docker images.
- Internet connection for accessing GitHub and downloading required packages.

4.2 Software Requirements

Software	Purpose
Python	Backend application development
HTML	Webpage structure
CSS	User interface styling
Git	Version control
GitHub	Online source-code repository
Docker	Application containerization
Docker Compose	Service configuration and management
pytest	Automated testing, if used by the project
Prometheus	Metrics collection and monitoring
Grafana	Metrics visualization
Web browser	Accessing the application
Code editor / terminal	Development and execution

4.3 System Description

The project consists of a backend application, a frontend interface, dependency definitions, automated tests, containerization files, and monitoring configuration.

The `app.py` file contains the Python application logic. The `templates/index.html` file provides the webpage structure, while `static/style.css` contains the styling rules. The `requirements.txt` file lists the required Python dependencies.

The `tests/test_app.py` file contains automated tests for application behavior. The `Dockerfile` defines the instructions for building a Docker image, and `docker-compose.yml` defines the application services. The `prometheus.yml` file provides configuration for Prometheus monitoring targets.

5. TECHNOLOGY STACK AND DEVOPS TOOLS USED

5.1 Python

Python is used to implement the backend logic of the application. It handles application requests and supports the functionality provided by the Todo List system.

5.2 HTML and CSS

HTML defines the structure of the application's webpage, while CSS controls the layout, styling, and appearance of the user interface.

5.3 Git and GitHub

Git tracks changes to project files and maintains version history. GitHub hosts the repository online, making the source code accessible for review and collaboration.

5.4 Docker

Docker packages the application and its dependencies into a container image. This helps maintain a consistent execution environment across systems.

5.5 Docker Compose

Docker Compose defines and manages the application's containers and supporting services using a YAML configuration file.

5.6 Prometheus

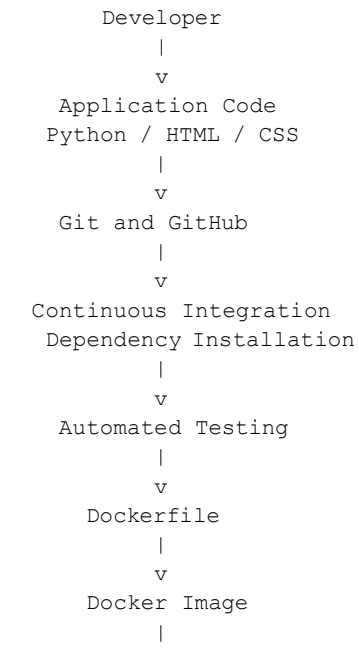
Prometheus collects and stores metrics from configured monitoring targets. It can help monitor service availability and application behavior when compatible metrics are exposed.

5.7 Grafana

Grafana is a visualization platform that can connect to Prometheus and display collected metrics through dashboards and panels.

6. SYSTEM / DEVOPS ARCHITECTURE DIAGRAM

The following diagram represents the intended workflow of the project.



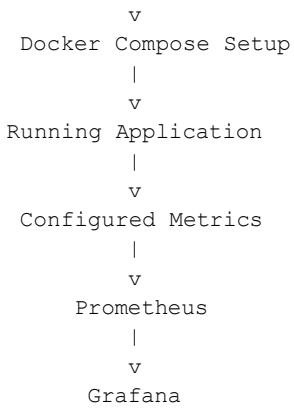


Figure 1: System and DevOps Architecture

The architecture illustrates how source-code management, testing, containerization, deployment, and monitoring can be connected in a DevOps workflow. Actual execution of each stage must be supported by the corresponding project evidence.

7. GIT AND GITHUB IMPLEMENTATION

Git and GitHub are used to maintain the project's source code and provide a centralized location for storing the project files.

The project repository contains the following files and directories:

- `app.py`
- `Dockerfile`
- `docker-compose.yml`
- `prometheus.yml`
- `requirements.txt`
- `README.md`
- `PROJECT_REPORT.md`
- `static/style.css`
- `templates/index.html`
- `tests/test_app.py`

7.1 Implementation Steps

1. Create a repository on GitHub.
2. Add the project source code and configuration files.
3. Commit changes to record the project version.
4. Push local changes to the remote repository when using Git locally.

5. Review the uploaded files on GitHub.
6. Maintain project instructions and documentation in the README file.

7.2 Common Git Commands

```
git add .
git commit -m "Update Todo List project"
git push
```

These commands illustrate how modified files can be committed and pushed to GitHub from an already configured local repository.

Result: The project files are available in the GitHub repository.

Evidence: Insert a screenshot of the GitHub repository showing the project files.

8. CI PIPELINE DESIGN AND CONFIGURATION

Continuous Integration (CI) is a DevOps practice in which code changes trigger automated checks. It helps developers detect errors early and verify that the application continues to work after modifications.

A CI pipeline for this project can be designed using GitHub Actions.

8.1 CI Pipeline Stages

1. **Source checkout:** Retrieve the latest project code from GitHub.
2. **Environment setup:** Install the required Python version.
3. **Dependency installation:** Install the packages listed in `requirements.txt`.
4. **Automated testing:** Execute the test suite.
5. **Result reporting:** Display whether the workflow passed or failed.

8.2 Example CI Workflow

The following configuration illustrates a possible GitHub Actions workflow. It should be added and tested only if the repository does not already have an equivalent workflow.

File path: `.github/workflows/ci.yml`

```
name: Todo List CI

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest

      - name: Run tests
        run: python -m pytest -v
```

The Python version and test command must match the project requirements. The existence of a CI configuration or a successful workflow run should be verified separately.

Evidence: Insert a screenshot of the GitHub Actions workflow and its actual execution result.

9. BUILD AND TESTING PROCESS WITH RESULTS

Testing is used to verify that the application behaves according to its requirements. Automated tests help identify defects and reduce the risk of errors during later development.

The project contains a test file named `tests/test_app.py`.

9.1 Testing Procedure

First, install the dependencies:

```
python -m pip install -r requirements.txt
```

Then run the automated test suite:

```
python -m pytest -v
```

The test runner displays the test cases executed and their results.

9.2 Test Cases

The reported test output covers the following areas:

Test case	Purpose	Reported result
Homepage test	Check the main application page	Passed
Add-task test	Check task creation functionality	Passed

Test case	Purpose	Reported result
Empty-task validation	Check rejection of empty task input	Passed
Health endpoint test	Check the application health endpoint	Passed
Metrics endpoint test	Check the metrics endpoint	Passed

Reported result: 5 tests passed.

This result should correspond to the final version of the submitted project. Include the actual terminal screenshot showing the five passing tests.

10. DOCKERFILE AND CONTAINERIZATION PROCESS

Containerization packages an application and its required dependencies into an isolated execution environment. Docker helps reduce differences between development and deployment environments.

10.1 Dockerfile

The project contains a `Dockerfile` that defines how the application image is built. The file specifies the base environment and the instructions required to prepare and start the application.

The exact build instructions should be understood from the submitted Dockerfile.

10.2 Building the Docker Image

Open a terminal in the project directory and execute:

```
docker build -t todo-list-app .
```

This command builds an image using the Dockerfile in the current directory.

10.3 Running the Container

The container can be started using:

```
docker run -p 5000:5000 todo-list-app
```

This is an example command. The port mapping must match the port on which the application listens.

10.4 Docker Compose

The project includes `docker-compose.yml`, which defines the configured application services.

To build and start the configured services, use:

```
docker compose up --build
```

To view running containers:

```
docker ps
```

To stop the Compose-managed services:

```
docker compose down
```

Result: Docker provides a container-based method of running the application and its configured supporting services.

Evidence: Insert screenshots of the Docker image build and running containers.

11. DEPLOYMENT PROCESS AND ENVIRONMENT DETAILS

Deployment involves making the application available in an environment where it can be accessed and tested.

11.1 Deployment Steps

1. Verify that the application source files and dependencies are available.
2. Install Docker and Docker Compose in the target environment.
3. Build the application image using the Dockerfile.
4. Start the configured services using Docker Compose.
5. Inspect the running containers and application logs.
6. Access the application through its configured address and port.
7. Verify the expected application functionality.

11.2 Deployment Environment

Parameter	Details
Application	Todo List web application
Backend	Python
Frontend	HTML and CSS
Containerization	Docker
Service configuration	Docker Compose

Parameter	Details
Environment	Local container environment, if confirmed by the screenshots
Application port	As configured in the project
Public deployment URL	Not specified

The available evidence relates to a local Docker-based environment. A public cloud deployment should not be claimed unless it has actually been performed.

Evidence: Insert a screenshot of the running application and the relevant container environment.

12. MONITORING SETUP USING PROMETHEUS AND GRAFANA

Monitoring helps developers observe application behavior, service availability, and operational metrics.

12.1 Prometheus Configuration

The project contains `prometheus.yml`, which is used to define Prometheus scraping settings and monitoring targets.

Prometheus periodically collects metrics from configured targets. For collection to work, the target must be reachable and expose metrics in a compatible format.

12.2 Grafana Configuration

Grafana can visualize metrics collected by Prometheus.

The general setup procedure is:

1. Start Prometheus and Grafana using the configured environment.
2. Verify that Prometheus can reach the metrics endpoint.
3. Check the target status in the Prometheus interface.
4. Add Prometheus as a data source in Grafana.
5. Create a dashboard using available metrics.
6. Verify that the dashboard displays actual metric data.

12.3 Metrics

Depending on the application's instrumentation, monitoring may include:

- HTTP request counts.

- Application health.
- Request response times.
- Service availability.
- Container resource usage when suitable exporters are configured.

The reported Grafana evidence includes a query using the metric `todo_http_requests_total`. This metric can be used to examine HTTP request counts if it is exposed and collected correctly.

Result: The project includes monitoring configuration and evidence of monitoring-tool usage. The final report should include actual Prometheus target status and Grafana output screenshots to demonstrate successful monitoring.

13. SCREENSHOTS / EVIDENCE

Screenshots provide visual proof of the implementation and execution of the project.

The following evidence should be included in the final report:

1. **GitHub Repository:** Shows the uploaded source files.
2. **GitHub Actions:** Shows the actual CI workflow result, if configured.
3. **Test Execution:** Shows the five passing test cases.
4. **Docker Build:** Shows the image build process.
5. **Docker Containers:** Shows the running application and configured services.
6. **Application Interface:** Shows the Todo List application in a browser.
7. **Prometheus:** Shows the configured targets and their actual status.
8. **Grafana:** Shows the metrics query or dashboard.

Each screenshot should be followed by a caption explaining what it demonstrates.

14. RESULTS AND DISCUSSION

The Todo List DevOps Mini Project organizes a Python-based web application with frontend files, dependency definitions, automated tests, containerization configuration, and monitoring configuration.

The reported test output indicates that five test cases passed, covering the homepage, task creation, empty-task validation, application health, and metrics endpoint. Docker configuration files support container-based execution, while the Prometheus configuration provides a foundation for monitoring.

The screenshots should be used to demonstrate the actual GitHub repository, testing, container activity, and monitoring setup. CI workflow status and monitoring health should be reported according to their verified execution results.

The project demonstrates how DevOps tools can support the development and operation of a web application.

15. ADVANTAGES

1. Provides a web-based application for managing tasks.
2. Maintains source code in a centralized GitHub repository.
3. Supports version tracking through Git.
4. Documents application dependencies.
5. Supports consistent execution using Docker.
6. Allows automated testing of application functionality.
7. Provides a foundation for continuous integration.
8. Supports monitoring through Prometheus and Grafana when configured.

16. LIMITATIONS

1. The application functionality is limited to the features implemented in the source code.
2. Automated testing covers only the test cases provided.
3. Container execution depends on correct configuration and available resources.
4. Monitoring requires valid metrics endpoints and reachable targets.
5. CI and deployment stages require their own verified configuration and execution evidence.
6. Cloud deployment and production readiness cannot be assumed from the repository files alone.

17. FUTURE SCOPE

The project can be enhanced in the following ways:

1. Add database storage for persistent task management.
2. Introduce user authentication and individual task lists.
3. Expand unit and integration test coverage.
4. Complete CI automation for build and testing.
5. Introduce a verified CI/CD pipeline for deployment.
6. Deploy the application to a cloud platform.
7. Add application health checks, logging, and alerts.
8. Develop Grafana dashboards for useful application metrics.
9. Improve security, error handling, and configuration management.

18. CONCLUSION

The Todo List DevOps Mini Project demonstrates the use of common software development and operations tools in a web application workflow. Python, HTML, and CSS are used for the application components, while Git and GitHub support source-code management. Docker and Docker Compose provide containerization and service configuration, and Prometheus and Grafana support the monitoring workflow when correctly integrated.

The reported test results indicate that five test cases passed. The project structure and available evidence provide a foundation for understanding version control, testing, containerization, deployment, and monitoring. Additional verification of the CI pipeline, deployment environment, and monitoring targets helps ensure that the final report accurately reflects the implemented system.

19. REFERENCES

1. Git Documentation: https://git-scm.com/doc
2. GitHub Documentation: https://docs.github.com/
3. Python Documentation: https://docs.python.org/3/
4. Docker Documentation: https://docs.docker.com/
5. Docker Compose Documentation: https://docs.docker.com/compose/
6. Prometheus Documentation: https://prometheus.io/docs/
7. Grafana Documentation: https://grafana.com/docs/
8. pytest Documentation: https://docs.pytest.org/

GitHub Repository: https://github.com/pratiksha-jadhav01/MiniProject